POWERODBA · IBM.POWER_AIX_ORACLE_DBA

Oracle 19c Cross-Endian Migration (M5)

Complete reference for the PowerODBA playbook and roles that automate Oracle 19c cross-endian database migration — x86 Linux to IBM AIX Power — using Oracle's M5 method (RMAN cross-platform incremental backups + Full Transportable Data Pump, orchestrated by Oracle's DBMIG 5.1 driver tool).

`MOS Doc 2999157.1` `DBMIG 5.1` `Oracle 19c EE ≥ 19.18`

`1 playbook · 3 roles · 34 task files` `3 topologies supported`

01 · OVERVIEW

What the migration is for

The M5 migration in the PowerODBA collection (`ibm.power_aix_oracle_dba`) is one playbook, `m5_migration.yml` , and three roles that automate a complete Oracle 19c database migration from x86 Linux to IBM AIX Power — end to end, as repeatable playbook runs instead of a manual, multi-day DBA runbook. It exists because that particular move crosses an endian boundary (x86 is Little-Endian, AIX Power is Big-Endian), which rules out the two normally-simplest migration paths — physical standby isn't supported across different endian platforms at all, and a plain export/import would work but is too slow for a real database. Oracle's supported answer to that specific problem is a documented procedure called **M5** (MOS Doc 2999157.1); this playbook *is* that procedure, automated.

Concretely, it:

- Validates both hosts and both databases against every M5 prerequisite before touching anything, and fails loudly with a specific remediation step rather than guessing
- Deploys and drives Oracle's own DBMIG 5.1 driver tool for the repeatable RMAN backup/restore cycles that pre-stage the data
- Runs the Data Pump Full Transportable cutover and verifies source-vs-destination table counts before it will let downtime end
- Supports three topologies out of the box — Non-CDB → Non-CDB, Non-CDB → PDB, and PDB → PDB — selected by four variables, with no separate code path to maintain per topology
- Is idempotent throughout: prechecks and the L0 baseline are safe to re-run, L1 is designed to run daily, and the irreversible go-live step is a deliberate, separate, opt-in command

Why incremental backups first

L0 (full baseline) and repeatable L1 (incremental) backups move nearly all of the data *before* the maintenance window opens. Only the L1F pass — the last, smallest incremental — happens during actual downtime.

Why Full Transportable

Data Pump Full Transportable Export/Import moves the schema/metadata across endianness in one pass, while the actual datafiles arrive via the RMAN restores that already ran — avoiding a slow row-by-row export of the whole database during downtime.

What DBMIG 5.1 does

Oracle's own driver script (`dbmig_driver_m5.sh`) generates the RMAN backup/restore commands and manages the transport-tablespace READ ONLY/READ WRITE flip. This collection deploys it, configures it, and drives it — it does not replace it.

THREE SUPPORTED TOPOLOGIES

Non-CDB → Non-CDB, **Non-CDB → PDB** (source is a standalone database, destination is a pluggable database inside a container database), and **PDB → PDB** (both ends are pluggable databases) — selected entirely by four variables (`mig_pdb` , `pdb_name` , `dest_cdb_sid` , `dest_pdb_name`). See Topology flows below for the exact mechanics of each.

Detailed walkthrough

The same seven-phase shape as any RMAN cross-platform incremental migration — but the specifics below are this collection's actual behavior, not the general method. A few things a generic XTTS writeup gets right in principle work differently here, called out inline.

1 Pre-migration checks

- Source version, edition, and endian format confirmed (Oracle ≥19.18.0.0.0, Enterprise Edition, Little-Endian source / Big-Endian destination)
- The tablespace list to transport is identified — everything except SYSTEM, SYSAUX, UNDO, and TEMP, which are never transportable
- `DBMS_TTS.TRANSPORT_SET_CHECK` runs against that list and must report zero violations — this is a **hard gate before the baseline backup is even allowed to start**, not something deferred to be patched up during import
- Character set, NCHAR character set, and timezone file version are asserted to match source and destination *exactly* — no automated fix exists for a mismatch here
- Source and destination do **not** need identical patch levels — only the same minimum version plus specific mandatory patches (the DataPump Bundle Patch on both sides, the RMAN cross-platform restore bugfix on the destination)

2 Database preparation

- Tablespaces stay READ WRITE through this entire phase — the source keeps running normally; READ ONLY is only needed for the last few minutes at final cutover
- Oracle's own DBMIG 5.1 driver and an identical working-directory tree are deployed to both hosts
- How backup pieces move from source to destination is set once, up front: either SCP, or an existing shared filesystem mount (`shared_filesystem: YES`) — this collection doesn't support `DBMS_FILE_TRANSFER` or rsync as alternatives

3 Baseline backup and restore (L0)

- Full RMAN backup of the transportable tablespaces on source, while still READ WRITE
- Backup pieces transferred to the destination
- Restored there — cross-platform datafile conversion happens automatically as part of the restore command itself (`RESTORE ... FOREIGN DATAFILES ... FROM BACKUPSET`), confirmed directly from a real generated cmdfile; there's no separate manual `CONVERT DATAFILE` step in this flow
- This restored copy becomes the baseline everything else builds on

4 Incremental sync (L1, repeated)

- Source keeps running and drifting ahead of the baseline
- One or more incremental backups capture only what changed since L0 (or the last L1)
- Each is restored, rolling the destination copy incrementally forward — run this daily, or on whatever cadence matches the real change rate, right up until the planned cutover
- Each cycle shrinks how much data the final cutover pass has to move

5

⚠ CRITICAL — DOWNTIME WINDOW

Final backup and restore (L1F) — the actual cutover step

- Transportable tablespaces set READ ONLY on source — **downtime starts here**
- The last incremental backup is taken and restored — the destination copy is now fully caught up
- Because nearly everything already moved in phases 3–4, this pass is small and fast

6

⚠ CRITICAL — THE REAL PASS/FAIL GATE

Metadata import

- Data Pump Full Transportable Import (`impdp` with `TRANSPORT_DATAFILES`) moves only the schema and metadata — the actual data is already on disk from the restores above
- This plugs the transported tablespaces into the destination database's data dictionary
- The real pass/fail gate here is a **source-vs-destination row-count comparison** — not a checksum comparison, and not `DBMS_TTS.TRANSPORT_SET_CHECK` again (that already ran, and only matters, before phase 3)

7

Post-migration and go-live

- Transported tablespaces set back to READ WRITE on the destination — **downtime ends here**
- Invalid objects recompiled, optimizer statistics refreshed
- Once verified and accepted as production, the guaranteed flashback restore point Data Pump created is dropped — a separate, explicit, opt-in step, since it's irreversible

OUTSIDE THIS COLLECTION'S SCOPE

Updating application connection strings/TNS entries, DNS or listener cutover, checksum-based data validation, and post-go-live monitoring are all real parts of a production migration — none of them are automated here. This collection stops at a verified, READ WRITE destination database with matching row counts; the application-facing cutover and ongoing monitoring are on you.

02 · BEFORE YOU START

Prerequisites

Three environments are involved, each with its own requirements. Nothing here is automated by the collection except where noted — these are conditions the collection *checks* (and fails loudly on) rather than fixes for you.

controller Ansible controller

Software

- ✓ Ansible >= 2.14, Python >= 3.9
- ✓ `ansible.posix` collection >= 1.4.0 (SSH key push)
- ✓ SSH client access to both the source and destination hosts
- ✓ `ansible-vault` password file (`~/.vault_pass`) — PowerODBA's `ansible.cfg` doesn't set one, so every command passes it

Files you stage yourself

- ✓ Oracle's `DBMIG_5.1.zip` is Oracle-proprietary and **not** distributed with PowerODBA. Download it from My Oracle Support (Doc 2999157.1) and place it at `roles/mig_prechecks/files/` — `deploy_dbmig.yml` copies it from there to both hosts. The repository's `.gitignore` keeps it out of git.
- ✓ Real passwords still need to be filled into `playbooks/vars/m5_migration/mig_vault.yml` before encrypting it — see Download & install.

THE DATAPUMP BUNDLE PATCH IS NOT STAGED OR INSTALLED BY THIS COLLECTION

Patch 38170982 is **not** a file anyone downloads or stages here, and this collection never runs `opatch apply` for it. It must already be **applied** — manually, by hand, on both the source and destination — before prechecks run. `patch_apply_source.yml` / `patch_apply_dest.yml` only run `opatch lsinventory` and assert the patch ID is found; if it isn't, prechecks fails with the exact ID to apply and stops there.

source Source database host (x86 Linux)

- ✓ Oracle 19c **Enterprise Edition** + Grid Infrastructure already installed and running, version >= 19.18.0.0.0
- ✓ Platform confirmed **Little Endian**
- ✓ Database in **ARCHIVELOG** mode
- ✓ RMAN `ENCRYPTION FOR DATABASE` must be **OFF** (not supported by this procedure)
- ✓ `oracle` OS user with SSH equivalence to the destination host (bidirectional)
- ✓ DataPump Bundle Patch 38170982 already **applied** (checked via `opatch lsinventory` — the collection does not download or install it)
- ✓ Sufficient free space under the working directory for RMAN backup pieces and the export dump file
- ✓ If TDE is in use: keystore keys exported and imported into the destination manually beforehand, and a completion marker touched on the **source host** at `done_dir/tde_keys_transferred.done` (the collection only verifies this happened — it does not perform the key transfer)

destination Destination database host (IBM AIX Power)

- ✓ Oracle 19c **Enterprise Edition** + Grid Infrastructure already installed and running, version >= 19.18.0.0.0, confirmed **Big Endian**
- ✓ Destination database already created and **OPEN** (non-CDB or CDB, per your topology) — this collection does not create the database itself
- ✓ Database in **ARCHIVELOG** mode
- ✓ `compatible` >= the source database's `compatible` value
- ✓ `NLS_CHARACTERSET`, `NLS_NCHAR_CHARACTERSET`, and the `V$TIMEZONE_FILE` version must **exactly match** the source — there is no automated fix for a mismatch here; it requires a database rebuild
- ✓ `db_create_file_dest` set to exactly one location (an ASM diskgroup or a filesystem path)
- ✓ `/bin/bash` present — the vendor's `impdp.sh` uses bash-only array syntax; AIX's default `/bin/sh` is a real POSIX/Korn shell and will not run it correctly
- ✓ DataPump Bundle Patch 38170982 already **applied** (checked, not installed, by the collection)
- ✓ Bugfix 35564253 (RMAN cross-platform restore fix) applied as a standalone patch — unless the installed Release Update is >= 19.27.0.0.0, which already bundles it
- ✓ If the source uses TDE: bugfix 30817822 applied (or RU >= 19.20 which bundles it)
- ✓ If migrating from a physical standby: bugfix 35675516 applied
- ✓ `oracle` OS user with SSH equivalence to the source host (bidirectional)

03 · SETUP

Download and install

The migration ships inside the PowerODBA collection, `ibm.power_aix_oracle_dba` on Ansible Galaxy — install it like any other collection, then supply the one piece Oracle doesn't let anyone redistribute.

1

Install from Galaxy

Install the PowerODBA collection the normal way:

```
ansible-galaxy collection install ibm.power_aix_oracle_dba
```

Or work from a clone of github.com/IBM/ansible-power-aix-oracle-dba. Either way, the migration lives in `playbooks/m5_migration.yml` and `roles/mig_*`.

2

Install collection dependencies

From the collection's `playbooks/` directory:

```
ansible-galaxy collection install -r ../collections/requirements.yml
```

Installs `ansible.posix`, which the migration needs for SSH key setup. All DBA/RMAN/Data Pump work runs via plain `ansible.builtin.shell` against `sqlplus` / `rman` / `impdp.sh` — no custom Python modules to build or ship.

3

Stage DBMIG

Download `DBMIG_5.1.zip` from My Oracle Support (Doc 2999157.1) and place it at `roles/mig_prechecks/files/DBMIG_5.1.zip`. `deploy_dbmig.yml` copies it from there to both hosts and unzips it. It isn't distributed with PowerODBA, and `.gitignore` keeps it out of the repository.

NOT STAGED HERE: THE DATAPUMP BUNDLE PATCH

Patch 38170982 is a different thing entirely — it's not a file in `files/` and this collection never applies it. It must already be applied by hand on both hosts before prechecks run; see Prerequisites.

4 Set up the vault

The migration has its own vault file, separate from PowerODBA's `vars/vault.yml`. From `playbooks/`, fill in the real passwords (SYS, SYSTEM) and encrypt it:

```
# edit vars/m5_migration/mig_vault.yml — set vault_sys_password and
# vault_system_password to real values

ansible-vault encrypt vars/m5_migration/mig_vault.yml
```

Then create the password file the runbook commands expect:

```
echo '<vault-password>' > ~/.vault_pass
chmod 600 ~/.vault_pass
```

5 Describe your environment

`playbooks/inventory.yml` is shared with the other PowerODBA playbooks, so the migration hosts sit in their own groups. Replace only the two host lines (`<server hostname> ansible_host=<IP or DNS name>`), keeping exactly one host in each group:

```
[mig_source]
source-linux-host ansible_host=333.33.33.333

[mig_destination]
dest-aix-host ansible_host=444.44.44.444

[mig:children]
mig_source
mig_destination

[mig:vars]
ansible_user=oracle
ansible_python_interpreter=/usr/bin/python3
```

`[mig:children]` lists group names, not hosts — leave it as is. `[mig:vars]` applies only to the two migration hosts: `ansible_user` is the account Ansible logs in as *on those hosts*. Host addresses come only from the inventory; the vars file derives them from each host's `ansible_host`.

`playbooks/vars/m5_migration/mig_vars.yml` — every migration parameter: SIDs, working directories, topology, tablespace list, credentials references, timeouts. See Variables file for the full list.

04 · RUNNING IT

How it's executed

Everything runs through one playbook, `playbooks/m5_migration.yml`, filtered by Ansible tags — one tag per stage. It calls each role directly; there's no `import_playbook` layer. Each stage is meant to be run as its own `nohup`'d background command **from the `playbooks/` directory**, like the other PowerODBA playbooks, logging to its own file, so a multi-hour RMAN backup or Data Pump import survives an SSH disconnect. Every command passes `-i inventory.yml` (PowerODBA's `ansible.cfg` doesn't name an inventory) and `--vault-password-file ~/.vault_pass` (it doesn't set a vault password file; `--ask-vault-pass` can't prompt under `nohup`).

INVENTORY CHECK

The playbook's first play runs on the controller and checks that the inventory has exactly one host in `[mig_source]` and one in `[mig_destination]`. If `-i inventory.yml` was left out or the groups are missing, it stops straight away with *"The inventory doesn't define the migration hosts. Run from the playbooks/ directory with -i inventory.yml..."*.

1 source + destination tag: prechecks run once

Prechecks

OS/DB validation, patch verification, SSH equivalence setup, source and destination DB preparation, DBMIG 5.1 deployment. Everything in this stage is a check-then-fix-manually or a one-time setup step — see mig_prechecks. It runs as **one play across `mig_source:mig_destination`** — every included task file carries its own host-group `when:`, so each host runs only its half — followed by a small play that marks the stage done. The idempotency gate is evaluated per host, against the same done-file on the source.

```
nohup ansible-playbook m5_migration.yml -i inventory.yml --tags prechecks \
      --vault-password-file ~/.vault_pass > /tmp/m5_01_prechecks.out 2>&1 &
```

CONTROLLER, FROM PLAYBOOKS/

PLAY 1 — HOSTS: MIG_SOURCE:MIG_DESTINATION

1	both	Create `done_dir` on the source and `precheck_report_dir` on the controller, then check for `done_dir/prechecks.done` on the source — if it exists, the whole stage is skipped with a message
2	source	`os_checks_source.yml` — endian, Oracle version, charset/timezone/compatible facts, edition, ARCHIVELOG, RMAN encryption off, TDE detection, network reachability, disk/memory
3	destination	`os_checks_dest.yml` — Oracle home, GI home (skipped when `dest_oracle_home_gi` is blank), DB open, version, ARCHIVELOG, compatible ≥ source, charset/NCHAR/timezone **exact match** against the source facts (read via `hostvars` — the source checks have already run), M5 working directories created, disk/memory, reverse ping to source
4	source	`patch_apply_source.yml` — assert DataPump BP 38170982 is present
5	destination	`patch_apply_dest.yml` — DataPump BP 38170982, bugfix 35564253 (skipped if RU ≥19.27), TDE/standby bugfixes (conditional)
6	both	`ssh_equivalence.yml` — the source runs the source→destination half, the destination runs the destination→source half: test, auto-push key if needed, re-test, assert
7	source	`db_prepare_source.yml` — streams_pool_size, Block Change Tracking (file path built from `ORACLE_BASE`, queried live with `orabase`), USERS tablespace report, DBMS_TTS self-containment check, M5DIR/DATA_PUMP_DIR, RMAN configuration
8	destination	`db_prepare_dest.yml` — db_create_file_dest, USERS → USERS_DEF replacement, streams_pool_size, M5DIR/DATA_PUMP_DIR, SYS/SYSTEM dictionary stats
9	both	`deploy_dbmig.yml` — included for the source only, delegating its destination half: M5 directory tree on both hosts, copies + unzips DBMIG_5.1.zip, templates

`dbmig_driver.properties`, writes the tablespace list

10	controller	`generate_report.yml` — writes `precheck_<source-host>.txt` to `precheck_report_dir`

PLAY 2 — HOSTS: LOCALHOST

1	source	Touch `done_dir/prechecks.done` on the source host (delegated) — only after the play above finished

2 source + destination `tag: rman_l0` `run once`

L0 — full baseline backup + restore

The first full RMAN backup on source, restored on destination. No downtime. Unlike prechecks, this is **one single play** targeting `mig_source:mig_destination` together — every task below is guarded by its own host-group `when:`, so it still executes cleanly split by host even though it's one play.

```
                                                    CONTROLLER, FROM PLAYBOOKS/
nohup ansible-playbook m5_migration.yml -i inventory.yml --tags rman_l0 \
      --vault-password-file ~/.vault_pass > /tmp/m5_02_rman_l0.out 2>&1 &
```

1	source	Idempotency gate: check for `done_dir/rman_l0.done` on the source host — if present, the whole cycle is skipped with a message
2	source	`check_lockfile.yml` — fail fast if DBMIG's own lock file exists
3	source	`run_backup.yml` — `dbmig_driver_m5.sh L0` via async/poll, output teed to a timestamped log, exit code asserted
4	source	`verify_backup_log.yml` — scan that run's log for ORA-/RMAN- patterns
5	source	`get_restore_cmdfile.yml` — locate the RMAN restore cmdfile DBMIG generated, store as a fact for the destination side
6	source	`sync_backup_pieces.yml` — parse this run's exact piece paths from the cmdfile, SCP them to destination (skipped entirely if `shared_filesystem: YES`)
7	destination	`run_restore.yml` — `rman target /` (non-CDB/PDB) or `rman target sys@...` (PDB→PDB) via async/poll, exit code asserted
8	destination	`verify_restore_log.yml` — same ORA-/RMAN- scan against the restore log
9	source	Touch `done_dir/rman_l0.done` on the source host (delegated — written once, keyed off the destination host having finished its restore)

3 source + destination tag: rman_l1 repeat until cutover

L1 — incremental backup + restore

Backs up only the blocks changed since the last L0/L1, restores them. No downtime. Run daily (or on whatever cadence fits the change rate) right up until the cutover window — each run shrinks the amount of data the final L1F pass has to move.

```
                                                    CONTROLLER, FROM PLAYBOOKS/
nohup ansible-playbook m5_migration.yml -i inventory.yml --tags rman_l1 \
      --vault-password-file ~/.vault_pass > /tmp/m5_02_rman_l1_$(date +%Y%m%d).out
2>&1 &
```

Identical task sequence to L0 above (steps 2–8 in that list — the same `mig_rman_L0_L1` role, just invoked with `rman_level: L1`), with two differences: **no idempotency gate** at the start (L1 is meant to run repeatedly), and **no done-file** written at the end. Each run's restore cmdfile and logs get their own timestamp, so multiple L1 runs leave multiple `restore_L1_*.log` files behind — `run_impdp.yml` later reads *all* of them, not just the latest.

4 source + destination tag: rman_l1f ⚠ downtime window

L1F — cutover

The final incremental backup (this is what sets the transportable tablespaces READ ONLY on source — **downtime starts here**), restore on destination, Data Pump Full Transportable import, table-count verification against source, tablespaces flipped back to READ WRITE on destination (**downtime ends here**), recompile, stats, validation report. Unlike prechecks and L0/L1, this runs as **separate plays** — source, then destination, then a small completion-marker play — and the destination play itself is internally divided into 4 numbered stages, each with its own start/complete banner in the log.

```
nohup ansible-playbook m5_migration.yml -i inventory.yml --tags rman_l1f \
      --vault-password-file ~/.vault_pass > /tmp/m5_03_cutover.out 2>&1 &
```

CONTROLLER, FROM PLAYBOOKS/

PLAY 1 — HOSTS: MIG_SOURCE

1	source	Interactive pause: *"⚠ Confirm DOWNTIME WINDOW"* — requires pressing Enter before anything runs. Under `nohup` there's no terminal, so Ansible logs *"Not waiting for response to prompt as stdin is not interactive"* and carries on — make sure every user is disconnected **before** launching the cutover
2	source	▶ **STAGE 1/4 START** — "sets transport TSs READ ONLY, DOWNTIME STARTS"
3	source	`check_lockfile.yml`
4	source	`run_backup.yml` — `dbmig_driver_m5.sh L1F` ; this is the exact command that flips the transport tablespaces to READ ONLY on source
5	source	`verify_backup_log.yml`
6	source	`get_restore_cmdfile.yml`
7	source	`sync_backup_pieces.yml` — copies this run's RMAN pieces to destination
8	source	`capture_expdp_file.yml` — finds the Data Pump export dump file DBMIG generated, copies it to destination's dump directory
9	source	✓ **STAGE 1/4 COMPLETE**

PLAY 2 — HOSTS: MIG_DESTINATION

1	destination	▶ **STAGE 2/4 START** — "L1F Restore"
2	destination	`run_restore.yml` — restores the foreign datafiles from the synced pieces

3	destination	`verify_restore_log.yml`
4	destination	✓ **STAGE 2/4 COMPLETE**
5	destination	▶ **STAGE 3/4 START** — "DataPump Full Transportable Import"
6	destination	`pre_import_checks.yml` — asserts restore logs and the dump filename are ready, confirms `impdp.sh` is present
7	destination	`run_impdp.yml` — patches `impdp.sh` in place, writes the `M5IMPDP` TNS alias, runs it in test mode then real run mode via `bash`
8	destination	`verify_import.yml` — informational impdp log scan only
9	destination	`verify_migration_correctness.yml` — **the real pass/fail gate**: source-vs-destination table counts, hard fail on mismatch, before anything below runs
10	destination	✓ **STAGE 3/4 COMPLETE**
11	destination	▶ **STAGE 4/4 START** — "Postchecks (read-write cutback, recompile, stats, validate)"
12	destination	`set_tablespaces_readwrite.yml` — **downtime ends here**
13	source	`restore_source_tablespaces.yml` — runs from the destination play via `delegate_to`, sets source tablespaces back to READ WRITE (best-effort)
14	destination	`recompile_objects.yml` — `utlrp.sql`
15	destination	`gather_stats.yml`
16	destination	`validate_migration.yml` — writes the final report on the controller
17	destination	✓ **STAGE 4/4 COMPLETE**, then a run-once *"CUTOVER COMPLETE"* message pointing at the go-live command
18	destination	Go-live include is reached but skipped — it only executes when `--tags golive_drop_restore_point` was itself the tag passed on the command line

PLAY 3 — HOSTS: LOCALHOST

1	source	Touch `done_dir/rman_l1f.done` on the source host (delegated)

5 destination tag: golive_drop_restore_point manual, opt-in, irreversible

Go live

Once the migration is verified and accepted as the new production database, this separate, explicit run finds the guaranteed flashback restore point `impdp.sh` created during import, shows you its name/SCN/creation time, pauses for a typed confirmation, then drops it. This step is **never** bundled into the cutover run above — see Operational notes for why that matters.

CONTROLLER, FROM PLAYBOOKS/

```
ansible-playbook m5_migration.yml -i inventory.yml --tags golive_drop_restore_point
--ask-vault-pass
```

1	destination	Query `V$RESTORE_POINT` for every `before_imp_%` restore point with `guarantee_flashback_database = 'YES'`, ordered newest first
2	destination	Assert exactly one was found — if zero or more than one, the run stops here and lists every one it found by name, SCN, and creation time rather than guessing
3	destination	Interactive pause — explicit warning that this permanently removes the flashback safety net, requires pressing Enter
4	destination	`DROP RESTORE POINT <name>;`
5	destination	Re-query and assert it's actually gone

IDEMPOTENCY

Prechecks and L0 each write a done-file under `done_dir` — `{{ m5_workdir }}/done` on the **source host**, not the controller — once they succeed; re-running the same tag afterwards prints an "already completed" message and does nothing. Delete the relevant done-file on the source to force a re-run. L1 has no done-file gate — it's designed to run repeatedly. L1F writes `rman_l1f.done` when it finishes, but nothing checks for it before a run.

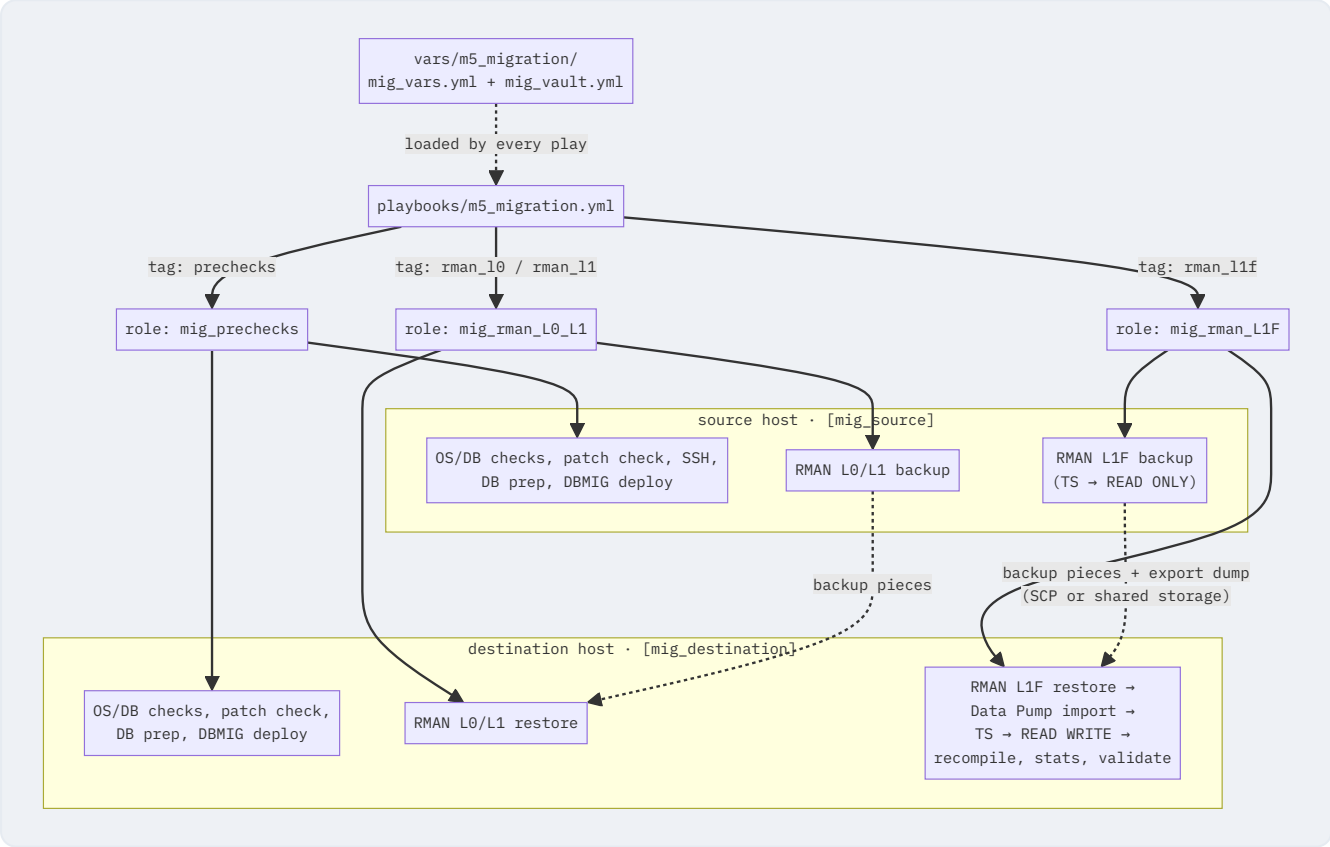
ONE SHOT, ALL STAGES

`ansible-playbook m5_migration.yml -i inventory.yml` with no `--tags` at all runs every stage in sequence in a single invocation — useful for a fully unattended test run, less useful for production where L1 needs to repeat over days and L1F needs a deliberately chosen maintenance window.

05 · ARCHITECTURE

Collection architecture

One playbook, three roles, one variables file, inside the PowerODBA collection. `m5_migration.yml` calls each role directly. Each role is tag-selected and host-scoped: prechecks and L0/L1 each run as a single play across `mig_source:mig_destination`, while L1F runs as one play per host group. Either way, tasks are guarded internally by `when: inventory_hostname in groups[...]`.



Every task file inside a role guards itself with a host-group check, so the same role definition serves both the source-side and destination-side halves of each stage.

Where it lives in PowerODBA

```
ansible-power-aix-oracle-dba/
├── .gitignore                      keeps roles/mig_prechecks/files/*.zip out of git
├── galaxy.yml                      collection metadata; depends on ansible.posix
├── collections/requirements.yml    ansible.posix
├── docs/
│   ├── Readme-M5-Migration.md      migration guide
│   └── Readme-M5-Migration.html    this reference
├── playbooks/
│   ├── ansible.cfg                 shared PowerODBA settings (unchanged by the migration)
│   ├── inventory.yml               shared inventory; migration hosts in the [mig] group
│   ├── m5_migration.yml            the migration playbook — one tag per stage
│   ├── roles -> ../roles           symlink PowerODBA playbooks use to find roles
│   └── vars/
│       └── m5_migration/
│           ├── mig_vars.yml        all migration settings
│           └── mig_vault.yml       encrypted: SYS / SYSTEM passwords
└── roles/
    ├── mig_prechecks/              OS checks, patch checks, SSH, DB prep, DBMIG deploy
    ├── mig_rman_L0_L1/             L0 baseline + L1 incremental (shared role, tag-
selected)
    └── mig_rman_L1F/               L1F + import + postchecks + validation (cutover)
```

06 · CONFIGURATION

Variables file — `vars/m5_migration/mig_vars.yml`

Every migration parameter lives in this one file — 55 variables, in the order below. Within each section mandatory variables come first and optional ones are marked *Optional*; derived values sit together in a final *Computed Values* block. Credentials reference `vars/m5_migration/mig_vault.yml` rather than holding plaintext. Fixed values that operators shouldn't change live in the roles' `defaults/main.yml` instead — see the second table.

VARIABLE	PURPOSE
DATABASE IDENTITY	
`source_oracle_sid`	Source database SID (renamed from `oracle_sid`)
`dest_oracle_sid`	Destination SID for non-CDB topology, or CDB root SID fallback
`listener_port`	Oracle listener port, both hosts — embedded in the destination and source connect strings
`source_oracle_home / dest_oracle_home`	$ORACLE_HOME on each host. The source $ORACLE_BASE is no longer configured — `db_prepare_source.yml` queries it with `orabase` when it builds the Block Change Tracking file path
`db_create_file_dest`	OMF destination for new datafiles (ASM diskgroup or path)
`dest_oracle_home_gi`	*Optional* — Grid Infrastructure home, verified via `crsctl`. Leave blank for a filesystem (JFS) destination with no ASM; the GI-home check is then skipped
HOST IDENTITY	
`dest_user / source_user`	OS user for SSH/SCP on each host (the oracle account). Host addresses are not set here — they come from the `[mig_source]` / `[mig_destination]` groups in `inventory.yml`; see `dest_host_effective`
FILE TRANSFER	
`shared_filesystem`	YES if source/destination share the same mounted working directory (no copy needed) — NO (default) copies backup pieces and the dump file via SCP

scp_transfer_timeout	*Optional* — ceiling (seconds) for the async backup-piece/dump-file transfer; only used when shared_filesystem is NO
MIGRATION WORKING DIRECTORIES	
done_dir	`{{ m5_workdir }}/done` on the **source host** — idempotency markers; delete a file here to force that stage to re-run
precheck_report_dir	Controller path for the `precheck_<host>.txt` reports
m5_workdir / m5_dumpdir / m5_backupdir	DBMIG working, Data Pump dump and RMAN backup directories — must be the identical absolute paths on both hosts
dest_workdir / dest_dpdmp	Same values as m5_workdir/m5_dumpdir, under DBMIG's own property names
validation_report_dir	Controller path for the final validation report — `/home/ansible/m5_migration/reports` by default, next to the precheck reports
TOPOLOGY (DEFINES MIGRATION TYPE — SEE TOPOLOGY FLOWS BELOW)	
mig_pdb	1 = source is a PDB (PDB→PDB), 0 = source is non-CDB
pdb_name	Source PDB name (mig_pdb=1 only)
dest_cdb_sid	Destination CDB root SID (Non-CDB→PDB and PDB→PDB only)
dest_pdb_name	Destination PDB name (Non-CDB→PDB and PDB→PDB only)
TABLESPACES TO MIGRATE	
m5_tablespace_list_items	List of source tablespace names to transport (all except SYSTEM/SYSAUX/UNDO/TEMP)
RMAN / DBMIG PARAMETERS	
rman_channels	Parallel RMAN channels DBMIG opens per backup/restore
rman_section_size	RMAN SECTION SIZE — splits large datafiles across channels
rman_timeout_l0 / l1 / l1f	Async ceiling (seconds) per backup/restore level
rman_poll	Check-in interval (seconds) for async RMAN jobs

`lock_file`	DBMIG's own concurrency lock path — a stale one blocks the next run until removed
`bkp_dest_type`	DISK or SBT_TAPE
`bkp_dest_parm`	Backup destination path (DISK mode)
`rman_log_benign_patterns`	*Optional* — short allow-list of informational `ORA-` codes (ships with ORA-28002, password-expiry warning) that must not fail the backup/restore log scan
DATAPUMP PARAMETERS	
`impdp_timeout / impdp_poll`	Async ceiling / check-in interval for the import job
`source_dpdmp`	Dump file location on source
`dp_parallel`	Data Pump parallel workers — must stay 1 below Oracle 21c (Full Transportable Tablespace jobs reject PARALLEL > 1 on 19c/20c)
`dp_enc_prompt`	*Optional* — Y/N, prompt for an encryption password
`dp_trace`	*Optional* — Data Pump trace level
CREDENTIALS (REFERENCE VARS/M5_MIGRATION/MIG_VAULT.YML)	
`sys_password`	SYS password — used only for the PDB→PDB RMAN restore connection
`system_password`	SYSTEM password — piped into both dbmig_driver_m5.sh and impdp.sh; the credential that actually matters for backup and import (the account name itself, `system_usr`, is a role default)
DBMIG MISC	
`remove_ts_quote`	1 = remove double-quotes around tablespace names in DBMIG's list
`bkp_from_stdby`	1 = source is a physical standby
`rman_debug`	Y/N — RMAN debug trace
`rman_scan`	*Optional* — destination SCAN Easy Connect (RAC only); leave blank for single-instance
`cat_cred`	*Optional* — RMAN catalog connect string (SBT_TAPE only)
DBMIG UTILITY	

dbmig_zip	Filename of the staged DBMIG_5.1.zip
DATAPUMP BUNDLE PATCH (DPBP)	
datapump_patch_id	38170982 — checked (never applied) on both hosts; version-specific to your Release Update (MOS KB107134)
APPLICATION SCHEMA	
app_schema	Application schema name used for stats gathering and the table-count correctness check
COMPUTED VALUES — DO NOT EDIT	
dest_host_effective	Destination address, taken from its `ansible_host` in inventory — used for Oracle Net connect strings, SSH/SCP and reachability checks. Replaces the old `dest_hostname` / `dest_ssh_host` pair (and `source_hostname` is gone too — the source is addressed the same way)
dest_oracle_sid_effective	CDB root SID when the destination is a CDB, else dest_oracle_sid
dest_connect_string_effective	`dest_host_effective:listener_port/<service>` — used for every "@connect_string" against the destination
src_scan	Source Easy Connect string DBMIG uses for the L1F read-only + export step — points at the PDB when mig_pdb=1
m5_tablespace_list	Comma-separated form of m5_tablespace_list_items, for DBMS_TTS

Role defaults — fixed values, not in mig_vars.yml

VARIABLE	PURPOSE
ROLES/MIG_PRECHECKS/DEFAULTS/MAIN.YML	
bugfix_patch_id	35564253 — RMAN cross-platform restore fix, destination only (skipped when RU ≥ 19.27)
tde_bugfix_patch_id / tde_bundled_min_version	30817822 / 19.20.0.0.0 — checked only when the source uses TDE
stdby_bugfix_patch_id	35675516 — checked only when bkp_from_stdby is 1
min_compatible_version	Lowest acceptable `compatible` setting

`min_disk_gb_source` / `min_disk_gb_dest` / `min_memory_gb`	Disk and memory thresholds the OS checks assert
`tde_keys_marker`	`{{ done_dir }}/tde_keys_transferred.done` — the marker you touch on the source after transferring TDE keys
`system_usr`	SYSTEM — the account DBMIG and impdp connect as
ROLES/MIG_RMAN_L1F/DEFAULTS/MAIN.YML	
`impdp_run_mode`	`run` — test | test-readonly | run | run-readonly; controls whether the import actually executes and whether tablespaces end READ WRITE
`expdp_dumpfile` / `restore_cmdfile`	*runtime-populated* — the export dump filename and the restore cmdfile path
`rman_level`	L1F
ROLES/MIG_RMAN_L0_L1/DEFAULTS/MAIN.YML	
`rman_level` / `restore_cmdfile`	L0 by default (each play sets L0 or L1) / runtime-populated cmdfile path

COMPUTED, NOT EDITABLE

`dest_host_effective`, `dest_oracle_sid_effective`, `dest_connect_string_effective` and `src_scan` are derived from inventory and the topology variables — every task in the collection that needs to reach the destination database uses one of these, never the raw `dest_oracle_sid` / `dest_pdb_name` directly. This is the single mechanism that makes the same task files work across all three topologies.

Roles & tasks reference

Three roles, 34 task files between them. Each role's `main.yml` is the sequencing spine — every other file is `include_tasks`'d from it in a fixed order, host-guarded internally.

roles/mig_prechecks/

mig_prechecks `tag: prechecks`

Runs once, gated by a done-file. Validates both hosts, verifies the mandatory patches are applied, sets up SSH equivalence, prepares both databases, and deploys DBMIG 5.1.

9 task files (+ 1 template) —

FILE	RUNS ON	RUNS / SKIPPED WHEN	PURPOSE
os_checks_source.yml	source	Whole role is skipped if `done_dir/prechecks.done` already exists. No internal skips otherwise — every check here is a hard assert.	Endian format, Oracle version ≥19.18, character set/timezone/compatible facts, Enterprise Edition check, ARCHIVELOG, RMAN encryption OFF, TDE detection (+ keystore-transfer marker gate), network reachability to destination, disk/memory space.
os_checks_dest.yml	destination	Same done-file gate, evaluated per host in the one combined play. The GI-home stat and assert are skipped when `dest_oracle_home_gi` is blank.	Oracle home + GI home presence, database OPEN, version ≥19.18, ARCHIVELOG, compatible ≥ source, character set / NCHAR / timezone exact-match asserts against source facts, M5 working directories created, disk/memory space, reverse connectivity to source.
patch_apply_source.yml	source	Same done-file gate. No other conditions — the assert always runs.	Check-only: asserts DataPump Bundle Patch 38170982 is present via `opatch lsinventory`. No installation performed.
patch_apply_dest.yml	destination	DataPump BP check always runs. Bugfix 35564253's *standalone-patch* assert is skipped when the installed RU is ≥19.27 (a debug message explains why instead); the TDE bugfix assert only runs if `tde_in_use` was detected on source; the standby bugfix assert only runs if `bkp_from_stdby: 1`.	Check-only: DataPump BP 38170982 (mandatory), bugfix 35564253 (version-gated), TDE bugfix 30817822 (conditional on source TDE use), standby bugfix 35675516 (conditional on `bkp_from_stdby`).

`ssh_equivalence.yml`	source + destination	Runs on both hosts — the source executes only the source→destination half, the destination only the destination→source half. Within each half, the key-push remediation step is skipped entirely if the initial passwordless-SSH test already succeeded.	Tests, auto-remediates (pushes public key) only if needed, re-tests, and asserts passwordless SSH for the oracle user in both directions.
`db_prepare_source.yml`	source	Same done-file gate. The Block Change Tracking `ALTER DATABASE ENABLE...` step — and the `orabase` lookup that builds its file path — is skipped if BCT is already enabled.	Sets streams_pool_size, disables audit_trail, enables Block Change Tracking if off, reports USERS tablespace contents, runs `DBMS_TTS.TRANSPORT_SET_CHECK` and asserts zero violations, creates the M5DIR/DATA_PUMP_DIR directory objects, configures RMAN (compressed backupset, parallelism, channel format).
`db_prepare_dest.yml`	destination	Same done-file gate. The entire USERS → USERS_DEF replacement block (steps 1–5) only runs if a USERS tablespace actually exists on the destination — otherwise a single "nothing to replace" message prints instead.	Sets/verifies db_create_file_dest is a single location, safely replaces the destination's USERS tablespace with a USERS_DEF substitute (moves tables + rebuilds indexes out, flips the default tablespace, drops USERS) so the transported USERS from source can land cleanly, sets streams_pool_size, creates M5DIR/DATA_PUMP_DIR, gathers SYS/SYSTEM dictionary stats.
`deploy_dbmig.yml`	source + destination	Only *included* for the source host (`when: inventory_hostname in groups['mig_source']`) — the destination host skips the include. It still reaches the destination host itself, though: every directory/copy task inside it runs a second time with `delegate_to: groups['mig_destination'][0]` .	Creates the identical M5 directory tree on both hosts, copies and unzips DBMIG_5.1.zip, templates `dbmig_driver.properties` , writes the tablespace list file.
`generate_report.yml`	source	Source host only, as the last step inside the done-file gate.	Writes `precheck_<source-host>.txt` to `precheck_report_dir` on the controller (version, charset, endian, patch status).

templates/dbmig_driver.properties.j2	template	Rendered once, by `deploy_dbmig.yml`, on the source host's pass through that file.	Renders DBMIG's own properties file — SID, PDB, connect strings, RMAN channels/section size, backup destination, DEST_SERVER (conditional on shared_filesystem).

roles/mig_rman_L0_L1/

mig_rman_L0_L1 `tags: rman_l0 · rman_l1`

One shared role for both baseline (L0) and incremental (L1) cycles — identical task flow, selected by the `rman_level` play variable. No downtime.

7 task files —

SHARED SKIP CONDITION

Every task below sits inside one of two blocks — the source-side backup block or the destination-side restore block — and both blocks are skipped entirely, in one shot, when `rman_level == 'L0'` *and* `done_dir/rman_l0.done` already exists. Running with `rman_level: L1` never hits this gate at all — L1 always executes.

FILE	RUNS ON	RUNS / SKIPPED WHEN	PURPOSE
check_lockfile.yml	source	Backup block only (see above). Fails the whole play immediately if the lock file is present — nothing after it runs.	Fails fast if DBMIG's own lock file exists (a previous run may still be active or died mid-run).
run_backup.yml	source	Backup block only.	Executes `dbmig_driver_m5.sh [L0|L1]` via async/poll, tees output to a timestamped log, asserts a clean exit code.
verify_backup_log.yml	source	Backup block only. The critical-error assert itself is skipped (not even evaluated) when the scan finds zero matching lines — a clean log short-circuits straight to the "log is clean" message.	Scans that run's own log (not the cumulative DBMIG log) for ORA-/RMAN- patterns; asserts none found.
get_restore_cmdfile.yml	source	Backup block only.	Locates the RMAN restore cmdfile DBMIG generated for this run (latest by mtime), stores it as a fact for the destination play to read via `hostvars`.

`sync_backup_pieces.yml`	source	Backup block. Internally, every copy/verify step here has its *own* second-level skip: all of them are bypassed (replaced by one "shared storage assumed" debug message) whenever `shared_filesystem: YES`.	Parses this run's exact backup piece paths out of the restore cmdfile (not a directory glob — avoids re-copying every historical piece), SCPs them to the destination with async/poll, verifies arrival.
`run_restore.yml`	destination	Restore block only. Which RMAN connect string gets used is decided by `mig_pdb` at template-render time, not a separate skip — both branches are mutually exclusive halves of one Jinja `{% if %}`.	Derives the destination-side cmdfile path DBMIG's own SCP delivered, runs `rman target /` (non-CDB/PDB via ORACLE_SID) or `rman target sys@...` (PDB→PDB) with async/poll, asserts a clean exit code.
`verify_restore_log.yml`	destination	Restore block only. Same "skip the assert on a clean scan" behavior as the backup-log check.	Same ORA-/RMAN- scan pattern as the backup-log check, against the restore log.

roles/mig_rman_L1F/

mig_rman_L1F `tag: rman_l1f`

The cutover role — combines final backup, restore, Data Pump import, and all post-migration DBA tasks into one downtime-window sequence, plus a separate opt-in go-live step.

18 task files —

NO OVERALL IDEMPOTENCY GATE

Unlike prechecks and L0, this role has **no done-file check** at the top — it's designed to be run once per real cutover, but nothing stops it from running again. What *does* gate individual tasks is a mix of host, play, and the conditions listed per row below.

FILE	RUNS ON	RUNS / SKIPPED WHEN	PURPOSE
`check_lockfile.yml`	source	Play 1 (source) only. Fails the whole play immediately if DBMIG's lock file is present.	Same stale-lock guard as the L0/L1 role.
`run_backup.yml`	source	Play 1 only, always. **Downtime starts here.**	Runs `dbmig_driver_m5.sh L1F`, which sets the transportable tablespaces READ ONLY on source before backing them up.
`verify_backup_log.yml`	source	Play 1 only. Critical-error assert skipped when the scan is already clean.	ORA-/RMAN- scan of the L1F backup log.

`get_restore_cmdfile.yml`	source	Play 1 only, always.	Locates the L1F restore cmdfile.
`sync_backup_pieces.yml`	source	Play 1 only. Every copy/verify step inside is bypassed when `shared_filesystem: YES`.	Copies this run's backup pieces to destination.
`capture_expdp_file.yml`	source	Play 1 only. Its dump-file *copy* step (not the find/fact steps) is skipped when `shared_filesystem: YES`.	Finds the Data Pump export dump file DBMIG generated, records its name/path/size as facts, copies it to the destination's dump directory.
`run_restore.yml`	destination	Play 2 (destination) only. Connect method (target / vs target sys@...) branches on `mig_pdb` at render time.	RMAN restore of the L1F pieces — same branch as L0/L1, using the topology-resolved connect variables.
`verify_restore_log.yml`	destination	Play 2 only. Assert skipped on a clean scan.	ORA-/RMAN- scan of the L1F restore log.
`pre_import_checks.yml`	destination	Play 2 only, always — this is a pure gate, no internal skips.	Asserts at least the L0 + L1F restore logs exist, resolves the dump filename from source facts, confirms impdp.sh is present.
`run_impdp.yml`	destination	Play 2 only. The script-patching and TNS-alias steps always run; the *test-mode* and *real-run* invocations of impdp.sh itself only fire when `impdp_run_mode` is `run` or `run-readonly` — with `test` / `test-readonly`, only the par file gets generated and nothing is actually imported.	Patches impdp.sh in place (ORACLE_HOME, ORACLE_SID, parallel/trace level, DATA_PUMP_DIR → M5DIR), writes a managed `M5IMPDP` TNS alias and points ORACLE_CONNECT_STRING at it, runs impdp.sh in test mode then in real run mode via `bash` (not `sh`) with the SYSTEM password piped to stdin. See Topology flows for exactly how the connect string differs per topology.
`verify_import.yml`	destination	Play 2 only, always runs — but it never fails the play regardless of what it finds (no hard assert here at all).	Informational only — scans the impdp log for ORA-/KUP-/IMP- lines (Full Transportable Import routinely logs benign noise) and looks up the guaranteed flashback restore point Data Pump created, for visibility.
`verify_migration_correctness.yml`	destination	Play 2 only, always runs. This is the one task in the whole cutover with a hard fail-on-mismatch assert — everything after it (READ WRITE, recompile, stats) never executes if it fails.	**The real pass/fail gate.** Compares application-schema table counts on source vs destination — before tablespaces go READ WRITE and before downtime is allowed to end.

`set_tablespaces_readwrite.yml`	destination	Play 2 only, reached only if the gate above passed. **Downtime ends here.** The `ALTER TABLESPACE ... READ WRITE` statement is issued only for tablespaces currently READ ONLY — anything already READ WRITE is left untouched (the statement isn't idempotent, so re-running it would error).	Checks current status first, flips only what needs flipping, then asserts every migrated tablespace is present and ONLINE.
`restore_source_tablespaces.yml`	source	Included from Play 2, but its SQL runs on the *source* host via `delegate_to`. Marked `ignore_errors: true` — a failed or unreachable source here does not fail the cutover, since the source may be intentionally shut down at this point per the runbook.	Sets the source's own tablespaces back to READ WRITE (best-effort).
`recompile_objects.yml`	destination	Play 2 only, always, once tablespaces are READ WRITE.	Runs `utlrp.sql`, reports invalid-object count before and after.
`gather_stats.yml`	destination	Play 2 only, always.	Gathers optimizer statistics for the application schema.
`validate_migration.yml`	destination	Play 2 only, always — the last task before the stage-complete banners.	Table counts, tablespace status, invalid-object count, allocated DB size (source vs destination), restore point name — all written to a final validation report on the controller.
`drop_restore_point.yml`	destination	**Only** when `golive_drop_restore_point` was itself the tag typed on the command line — checked against the `ansible_run_tags` magic variable, not just tag inheritance, specifically because this role is included with `tags: [rman_l1f]` and that alone would otherwise let it slip through during a normal cutover (confirmed live; fixed for exactly that reason). Never runs during an ordinary `--tags rman_l1f` run.	**Separate, opt-in only.** Finds the guaranteed restore point, asserts exactly one exists (lists full details and refuses to guess if there's more than one), pauses for a typed confirmation, drops it, verifies it's gone.

08 · TOPOLOGY FLOW

Non-CDB → Non-CDB

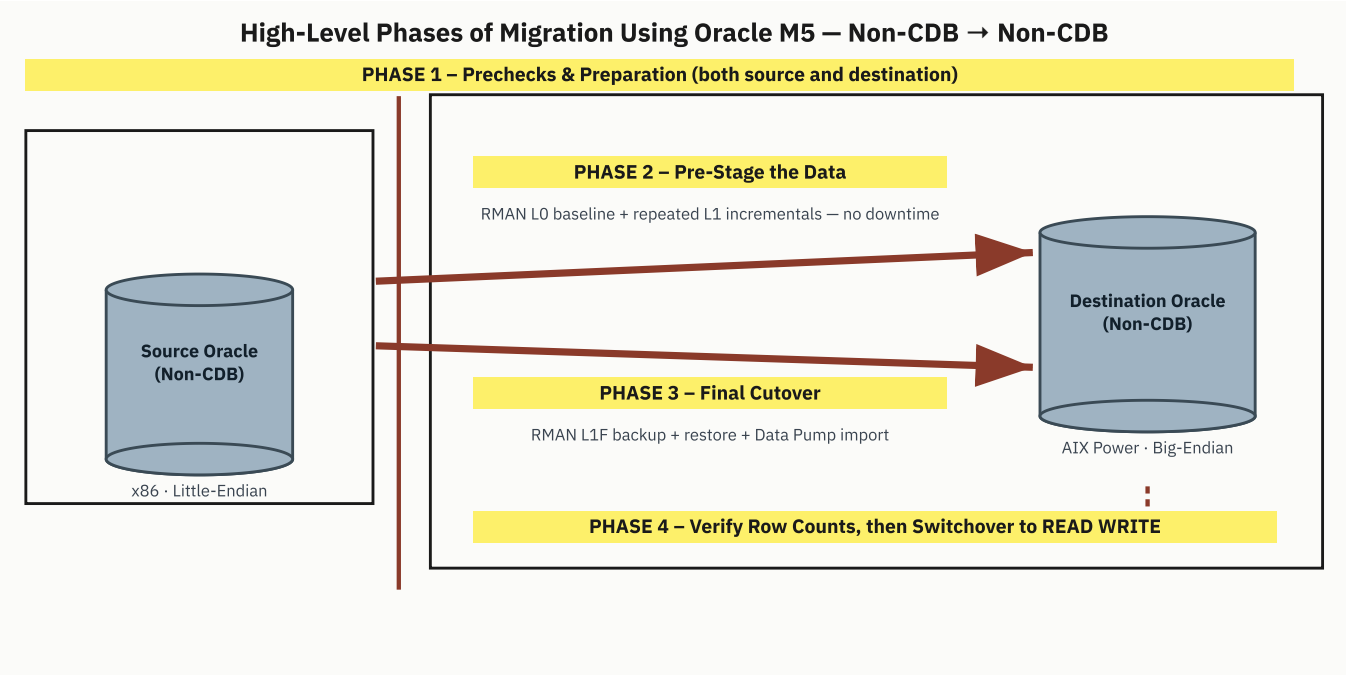
Both source and destination are plain, standalone (non-container) databases. This is the simplest case — every connection is local or a straightforward Easy Connect to a database SID, no container-switching involved.

• source source host • destination destination host • controller runs on the Ansible controller

Topology variables

```
mig_pdb:       0
pdb_name:      ""
dest_cdb_sid:  ""
dest_pdb_name: ""
```

Both computed variables fall all the way back to the plain SID: `dest_oracle_sid_effective` = `dest_oracle_sid`, and `dest_connect_string_effective` = `dest_host_effective:listener_port/dest_oracle_sid`, where `dest_host_effective` is the destination's `ansible_host` from inventory.



One plain database on each side — restore and import both connect straight to it, no intermediate container to reach through.

STAGE	RMAN RESTORE CONNECTS AS	DATA PUMP CONNECTS AS
Restore (L0/L1/L1F)	`target /` `(ORACLE_SID=dest_oracle_sid)`	—
Import	—	`system@M5IMPDP` `alias → dest_host_effective:port/dest_oracle_sid`

08 · TOPOLOGY FLOW

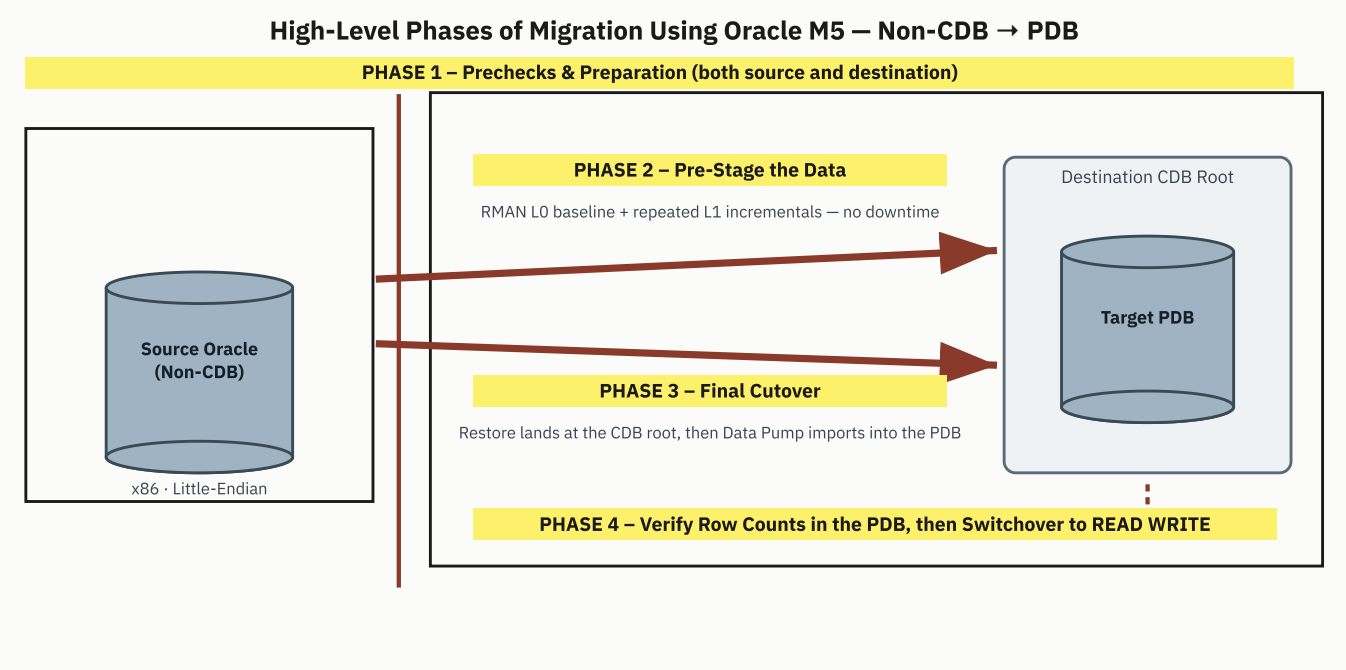
Non-CDB → PDB

Source is a standalone database; the destination is a pluggable database inside a container database. RMAN restore still connects locally to the CDB root by SID — it's Data Pump and every SQL*Plus task touching PDB-level objects that switch container.

Topology variables

```
mig_pdb:       0             # source is non-CDB → MIG_PDB=0 in DBMIG's properties
pdb_name:      ""             # no source PDB name
dest_cdb_sid:  "<CDB_SID>"    # destination CDB root SID
dest_pdb_name: "<PDB_NAME>"   # destination PDB name
```

`dest_oracle_sid_effective` = `dest_cdb_sid` (the root).
`dest_connect_string_effective` = `dest_host_effective:listener_port/dest_pdb_name` (the PDB's own service).



The destination is one database instance (a container database) with the real target — a pluggable database — living inside it. Restore lands the physical data at the container level; everything from import onward reaches into the pluggable database specifically.

STAGE	RMAN RESTORE CONNECTS AS	DATA PUMP / PDB-LEVEL SQL CONNECTS AS

Restore (L0/L1/L1F)	`target / (ORACLE_SID=dest_cdb_sid)`	—
Import & postchecks	—	`system@M5IMPDP alias → dest_host_effective:port/dest_pdb_name`

ONLY DIFFERENCE FROM NON-CDB → NON-CDB

The RMAN restore side is *identical* to the non-CDB case (still a local `target /` at CDB root) — only the SID it uses (root, not a plain SID) and everything downstream of the restore (Data Pump, container-scoped SQL) differ.

A DETAIL THAT TURNS OUT NOT TO MATTER

`run_impdp.yml` also exports `ORACLE_PDB_SID=<dest_pdb_name>` in the shell environment right before invoking `impdp.sh`, whenever `dest_pdb_name` is set. Reading the deployed script directly shows it never actually references that variable anywhere — the real PDB targeting is entirely carried by `ORACLE_CONNECT_STRING` (the M5IMPDP alias). The export is harmless and kept for parity with the KA documentation, but it isn't load-bearing.

08 · TOPOLOGY FLOW

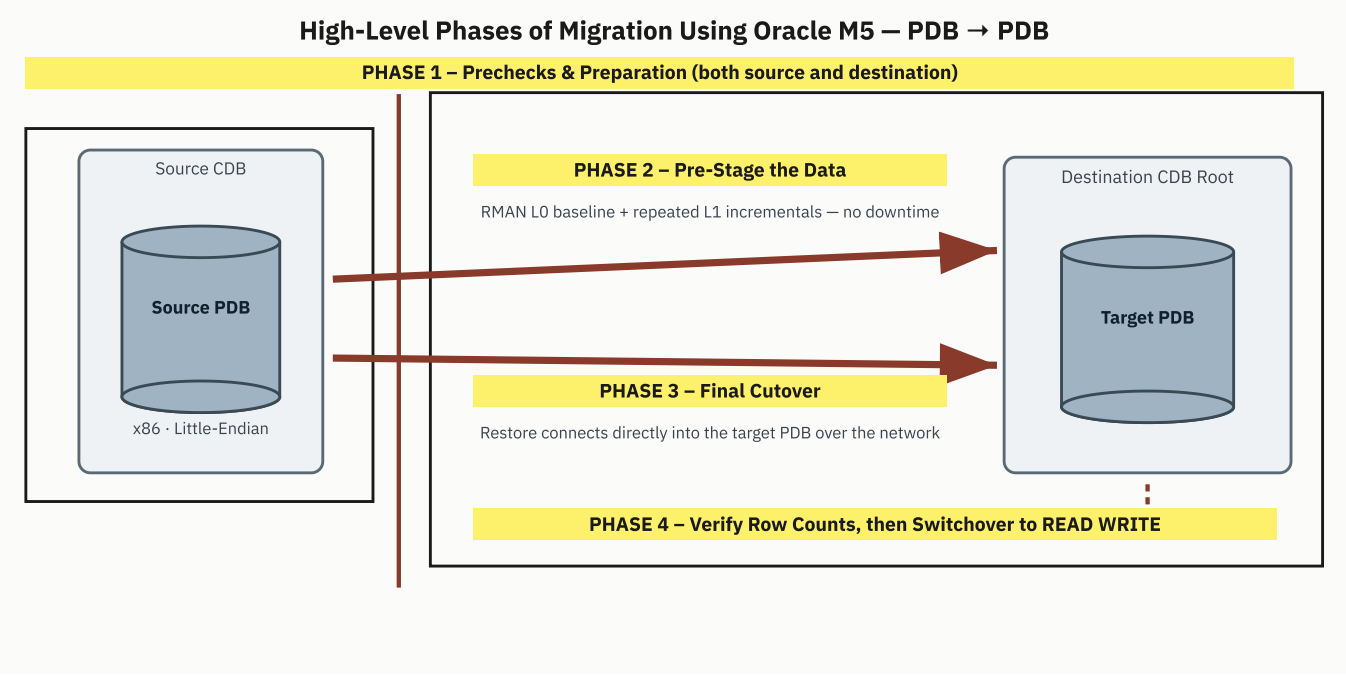
PDB → PDB

Both source and destination are pluggable databases. This is the only topology where the RMAN restore itself needs a real network connection with credentials, rather than a local bequeath connection — because the PDB being restored into isn't reachable via a bare local SID.

Topology variables

```
mig_pdb:       1
pdb_name:      "<SOURCE_PDB>"   # source PDB name — goes into DBMIG's PDB_NAME
property
dest_cdb_sid:  "<CDB_SID>"       # destination CDB root SID
dest_pdb_name: "<DEST_PDB>"      # destination PDB name
```

Same computed values as Non-CDB → PDB (`dest_oracle_sid_effective` = `dest_cdb_sid` , `dest_connect_string_effective` = the PDB's connect string) — but `mig_pdb=1` now changes how RMAN itself connects.



The only real difference from Non-CDB → PDB above is phase 4: since the source is also a pluggable database rather than a plain one, there's no bare local shortcut to restore through — RMAN has to reach the target pluggable database over the network instead.

STAGE	RMAN RESTORE CONNECTS AS	DATA PUMP CONNECTS AS

Restore (L0/L1/L1F)	target sys/<sys_password>@dest_connect_string_effective (PDB service — real network + credential connection)	—
Import	—	system@M5IMPDP alias → same dest_pdb_name service

WHERE THE BRANCH ACTUALLY LIVES

`run_restore.yml` (both the L0/L1 role and the L1F role) contains the only real conditional: `{% if mig_pdb | int == 1 %} rman target sys/<password>@... {% else %} rman target / {% endif %}`. Everything else — Data Pump's connect string, container-switching SQL, table-count verification — is already identical to Non-CDB → PDB, because both share the same PDB-shaped destination.

09 · OPERATIONAL NOTES

Things worth knowing before you run this for real

Drawn directly from real test runs against all three topologies — not hypothetical edge cases.

Restore points accumulate across retries

Every successful `impdp.sh run` mode creates a new guaranteed flashback restore point named `before_imp_<timestamp>`. If a cutover attempt fails partway and you re-run L1F, each attempt leaves its own restore point behind. The go-live drop step deliberately refuses to guess which one to drop once there's more than one — it lists all of them (name, SCN, creation time) and asks you to clean up manually via SQL*Plus first.

TNS alias name collisions

The collection writes a managed `M5IMPDP` alias into `tnsnames.ora` to work around a Data Pump connect-string parsing limitation. If a *different*, unmanaged entry with the same alias name already exists elsewhere in that file (e.g. left over from earlier manual testing), Oracle's resolver can pick the wrong one — with no error, just a silent wrong-database connection. Check for duplicate alias names if imports seem to be hitting stale data.

AIX's default shell is not bash

`impdp.sh` uses bash-only array syntax. AIX's `/bin/sh` is a genuine POSIX/Korn shell — running the script under it fails past the argument-parsing stage with no fatal error, silently imports nothing, and still reports success (rc=0). The collection always invokes it via `bash impdp.sh` explicitly.

Full Transportable Import logs benign noise

Errors like `ORA-39083` against the internal `GSMROOTUSER` account, or "object already exists" for tablespaces/directories, show up on essentially every run and are not failures. `verify_import.yml` only ever surfaces these for context — `verify_migration_correctness.yml`'s source-vs-destination table count comparison is the actual pass/fail signal.

An L1 with zero source-side changes is a hard stop, not a silent no-op

If nothing changed on source between the last backup and this L1 run, RMAN produces a genuinely empty incremental. Since there's nothing to copy or restore, the piece-sync step correctly finds zero pieces and fails loudly rather than pretending to succeed. This is expected on a quiet test database — make a real data change between L0/L1 runs to exercise the incremental path meaningfully.

Character set, NCHAR, and timezone mismatches have no automated fix

Unlike almost every other precheck, a mismatch here (`os_checks_dest.yml`) means a database rebuild on whichever side is wrong — there's no ALTER SYSTEM or patch that resolves it. Confirm these three match before investing time in the rest of the runbook.

A genuine non-CDB destination is uncommon

Most Oracle 19c installs default to the multitenant (CDB) architecture. If you point the automation at a CDB's root container while leaving the topology variables blank (Non-CDB → Non-CDB), Data Pump will reject creating ordinary local users at the root (`ORA-65096` , `ORA-65046`) — that combination only works when the destination instance was genuinely created without pluggable databases at all.

shared_filesystem controls a lot more than it looks

Beyond skipping the SCP steps, `shared_filesystem: YES` also blanks DBMIG's own `DEST_SERVER` property (so DBMIG doesn't redundantly SSH-copy files that are already visible on shared storage) — get this one setting right for the environment before the first L0 run, since it changes several tasks' behavior at once.

Clock skew across hosts can mislead file-based debugging

If the source, destination, and controller aren't NTP-synced, comparing file modification times across hosts to reconstruct "what ran when" can point at the wrong sequence of events. Database timestamps (SCN, restore point creation time) are the reliable source of truth when diagnosing a run after the fact.

A failed restore's log can outlive the failure — and this collection doesn't currently guard against that

`run_impdp.yml` deliberately unions *every* `restore_L0_*.log` / `restore_L1_*.log` / `restore_L1F_*.log` file it finds on disk (not just the latest per level), because L1 is meant to run repeatedly. That design doesn't currently distinguish a failed attempt's log from a successful one — a restore that fails partway still leaves its log sitting on disk under its normal name, so a later, separate L1F run can pick it up alongside the successful retry's log. If that failed attempt recorded a datafile path that's no longer valid on the retry (for example, if the datafile copy it referenced was deleted and re-created at a new path during cleanup), both paths get fed to `impdp.sh` 's `TRANSPORT_DATAFILES` list — one of them stale. This is a real, currently-unaddressed gap, not a hypothetical: if an L0, L1, or L1F restore fails and you retry it, check `{{ m5_workdir }}/log/` on the destination for that failed attempt's log file before running L1F's import, and move or rename it out of the way if there's any doubt about whether its recorded paths are still valid.

Run from `playbooks/`, with `-i inventory.yml` and the vault password

PowerODBA's `ansible.cfg` is shared and is found from the `playbooks/` directory. It names no inventory and no vault password file, so every migration command passes `-i inventory.yml` and `--vault-password-file`. Leave out the inventory and the playbook's first play stops with a clear message; leave out the vault password and even `--syntax-check` fails with *"Attempting to decrypt but no vault secrets found"*.

A full FRA quota stalls the database mid-run

When `+FRA` reached 100% of `db_recovery_file_dest_size`, archiving stopped and sessions hung on *"log file switch (archiving needed)"* — visible first as prechecks freezing on "Set USERS_DEF as database default tablespace". The diskgroup had plenty of free space; the quota was the limit. Check `v$recovery_file_dest` on both databases before L0 and size the quota for the archive logs a long baseline generates.

Done-files live on the source, and they're the only re-run switch

`done_dir` is `{{ m5_workdir }}/done` on the source host. If `prechecks.done` goes missing, the next `--tags prechecks` run re-runs every precheck for real — including the destination USERS → USERS_DEF replacement — instead of printing "already completed". Clearing `m5_workdir` during a reset also clears these markers, and the TDE key-transfer marker with them.

PDB→PDB restore errors are hidden from Ansible's output

`run_restore.yml` sets `no_log` when `mig_pdb` is 1, because the RMAN connect string carries the SYS password. A failed restore therefore shows only as *"censored"*. The real error is in the restore log under `{{ m5_workdir }}/log` on the destination — that's where errors like `ORA-17502: ksfdcre:4 Failed to create file` appeared during testing.

The shared `ansible.cfg` applies to the migration too

The migration sets its own login user and Python interpreter in `[mig:vars]` and changes nothing in `ansible.cfg`. Keep `remote_tmp = ~/.ansible/tmp` there: a shared path such as `/tmp/.ansible` fails as soon as another user owns it on a host (see Troubleshooting). Output format, per-task timings and a separate Ansible log aren't set; add them for a single run with `ANSIBLE_STDOUT_CALLBACK=yaml`, `ANSIBLE_CALLBACKS_ENABLED=timer,profile_tasks` or `ANSIBLE_LOG_PATH=…`.

10 · TROUBLESHOOTING

When something goes wrong

Real failures hit during testing of this collection, in Symptom → Cause → Fix form. A few are historical bugs already fixed in this collection — included anyway in case you're comparing against an older copy or hit something that looks similar.

Recovering from a failed run — full clean, then restart from Phase 1

This collection doesn't attempt any partial recovery, and doesn't clean up after itself. The recommended policy is deliberately simple rather than clever: if **L0, L1, or L1F fails for any reason**, don't try to resume mid-way — clean up everything below and restart the whole migration from prechecks. A targeted, partial cleanup can look correct and still leave something subtly stale (a leftover RMAN datafile copy, an old restore log with a now-invalid path); a full reset avoids that risk entirely.

`prechecks` — the one exception

Every task here is a pure check or an idempotent setup step (`CREATE OR REPLACE`, `ALTER SYSTEM`) — nothing destructive happens. Fix whatever the assert reported and re-run `--tags prechecks` directly, no cleanup needed.

Any L0, L1, or L1F failure — full reset

Regardless of which of the three failed, or at which internal step:

1 **destination** Clear the RMAN repository — a failed or partial restore can leave incomplete datafile copies behind:

```
rman target /

CROSSCHECK DATAFILECOPY ALL;
LIST DATAFILECOPY ALL;
-- review the list, then:
DELETE DATAFILECOPY ALL;
```

2 **both hosts** Remove everything under the M5 working directory (`{{ m5_workdir }}` , `/u01/m5` by default) — every backup piece, cmdfile, and log from every attempt lives here, and none of it is safe to assume is still valid after a failure:

```
rm -rf {{ m5_workdir }}/*
```

Prechecks re-stages everything this needs (DBMIG_5.1.zip, the directory tree, the properties file) on its next run, so there's nothing to preserve here. This also deletes `done_dir` , which lives under `m5_workdir` on the source — clearing the `prechecks.done` and `rman_l0.done` gates, which is exactly what a reset needs. It clears the TDE key-transfer marker too, so re-touch that before prechecks if TDE is in use.

3 **controller** Clean up the controller side:

```
rm -f /tmp/m5_*.out
rm -f {{ precheck_report_dir }}/*
rm -f {{ validation_report_dir }}/*
```

The done-files aren't here any more — step 2 already removed them with `m5_workdir` . This step is hygiene: clearing old logs and reports avoids confusing an old run's output with the new attempt.

4 **restart** Re-run from the top: `--tags prechecks` , then `rman_l0` , then `rman_l1` as needed, then `rman_l1f` .

go-live

Different case — this only reads `V$RESTORE_POINT` and pauses for a typed confirmation before doing anything destructive, so it's always safe to retry without any cleanup. See the "finishes but drops nothing" and "Found 2 … expected exactly 1" entries below for the two ways it can stop short.

Specific failures seen in testing

Import reports success but nothing was actually imported

SYMPTOM `impdp.sh` exits with rc=0, but the destination ends up with 0 tables and no guaranteed restore point.

CAUSE The script was invoked as `sh impdp.sh`. AIX's `/bin/sh` is a real POSIX/Korn shell and can't parse the script's bash-only array syntax — it fails partway through silently and never populates the file list, but still exits 0.

FIX Always invoke it as `bash impdp.sh ...` — already this collection's default. If a custom change reintroduces `sh`, this is the first thing to check.

Import fails with ORA-65096 or ORA-65046

SYMPTOM `ORA-65096: invalid common user or role name` or `ORA-65046: operation not allowed from outside a pluggable database` during import.

CAUSE The destination is actually a container database, but the topology variables were left blank as if it were Non-CDB → Non-CDB — Data Pump ended up connecting to the CDB root, which refuses to create ordinary local users.

FIX Set `dest_cdb_sid` and `dest_pdb_name` to target the real pluggable database (Non-CDB → PDB or PDB → PDB topology) instead.

Import connects to the wrong database

SYMPTOM `tnsping M5IMPDP` resolves to a database you didn't expect, or the import silently touches old/unrelated data.

CAUSE A second, unmanaged `M5IMPDP` entry already existed in `tnsnames.ora` — from earlier manual testing — outside the collection's own managed block. Oracle's resolver picked that stale entry instead.

FIX Search `tnsnames.ora` for more than one `M5IMPDP =` entry and remove the one outside the `ANSIBLE MANAGED BLOCK` markers.

L1 fails with "No backup piece paths found"

SYMPTOM The L1 or L1F backup-piece sync step fails: *"No backup piece paths found in ... cmdfile parsing may need review."*

CAUSE Usually not a parsing bug — nothing changed on the source between the last backup and this incremental, so RMAN genuinely produced a zero-byte backup with nothing to copy.

FIX Make a real data change on source (insert/update, then commit) before re-running L1. Rarely an issue in production, where there's normally ongoing activity between runs.

Go-live command finishes but drops nothing (ok=1, no error)

SYMPTOM `--tags golive_drop_restore_point` completes cleanly with no failures, but the restore point is still there.

CAUSE *(historical)* — Ansible doesn't propagate tags from a dynamic `include_tasks` statement into the included file's own tasks. The include fired correctly, but every task inside it was then filtered out as untagged.

FIX Already fixed in this collection — the tasks are wrapped in a block explicitly tagged `golive_drop_restore_point`. If you're on an older copy without that fix, add the block tag.

Go-live command refuses to proceed: "Found 2 ... expected exactly 1"

SYMPTOM The go-live step stops with a message listing two or more restore points instead of dropping one.

CAUSE Repeated or partially-failed L1F attempts each created their own guaranteed restore point, and none of the earlier ones were ever cleaned up.

FIX The error message itself lists every restore point (name, SCN, creation time) — drop the ones you don't want via SQL*Plus as `sys`, then re-run once only one remains.

Guaranteed restore point exists but the collection reports "none captured"

SYMPTOM `SELECT * FROM V$RESTORE_POINT` clearly shows a `BEFORE_IMP_...` row, but `verify_import.yml` and the validation report say no restore point was found.

CAUSE *(historical)* — the lookup query matched against a lowercase `'before_imp_%'` pattern, but Oracle stores unquoted restore point names in uppercase, and `LIKE` is case-sensitive.

FIX Already fixed — the query now uses `UPPER(name) LIKE 'BEFORE_IMP_%'`.

Prechecks fails on character set, NCHAR, or timezone mismatch

SYMPTOM Prechecks stops with a hard assert comparing source vs destination charset, NCHAR charset, or timezone file version.

CAUSE Source and destination were created with genuinely different settings — this isn't something Full Transportable Import can reconcile in flight.

FIX No `ALTER SYSTEM` fixes this. Timezone version can be upgraded via `DBMS_DST` on the lagging side; a character set mismatch requires a database rebuild. Confirm all three match before investing further time.

Setting tablespaces READ WRITE fails with ORA-01646

SYMPTOM The postcheck stage fails partway through with `ORA-01646`, and only some tablespaces end up READ WRITE.

CAUSE *(historical)* — `ALTER TABLESPACE ... READ WRITE` isn't idempotent; the task used to issue it unconditionally, and it errors on a tablespace that's already READ WRITE (like a pre-existing USERS).

FIX Already fixed — the task checks each tablespace's current status first and only alters the ones actually READ ONLY.

Import log shows GSMROOTUSER errors or "object already exists"

SYMPTOM `ORA-39083` against `GSMROOTUSER`, or "already exists" messages for tablespaces/directories, appear in the import log.

CAUSE Benign noise from Full Transportable Import — `GSMROOTUSER` is Oracle's own internal Global Data Services account, and "already exists" objects are expected on re-runs.

FIX Nothing to do — these show up on essentially every run. Check `verify_migration_correctness.yml`'s table-count result for the real pass/fail signal, not the raw log.

"The inventory doesn't define the migration hosts"

SYMPTOM The first play, *"M5 migration — check inventory"*, fails and nothing else runs.

CAUSE The playbook was run without `-i inventory.yml`, or `[mig_source]` / `[mig_destination]` are missing or don't hold exactly one host each.

FIX `cd` to `playbooks/`, run `ansible-playbook m5_migration.yml -i inventory.yml …`, and keep exactly one host in each migration group.

"Attempting to decrypt but no vault secrets found"

SYMPTOM Any command — even `--syntax-check` — stops with this error.

CAUSE `mig_vault.yml` is encrypted and PowerODBA's `ansible.cfg` doesn't name a vault password file.

FIX Add `--vault-password-file ~/.vault_pass`, or `--ask-vault-pass` when running interactively.

"Failed to create temporary directory" — reported as UNREACHABLE

SYMPTOM A host fails on its first task with *UNREACHABLE ... Failed to create temporary directory*, although SSH works.

CAUSE `remote_tmp` points at a directory the oracle user can't write to — typically a shared path such as `/tmp/.ansible` that another user (for example root, mode `drwx------`) created first.

FIX Keep `remote_tmp = ~/.ansible/tmp` in `ansible.cfg`, or give the migration hosts their own by adding `ansible_remote_tmp=~/.ansible/tmp` to `[mig:vars]` in `inventory.yml`.

Prechecks hangs on "Set USERS_DEF as database default tablespace"

SYMPTOM The task never returns; `v$session` shows sessions waiting on *"log file switch (archiving needed)"*.

CAUSE The FRA hit its `db_recovery_file_dest_size` quota (`v$recovery_file_dest` at 100% used, nothing reclaimable), so archiving stopped and anything needing a log switch waits.

FIX Raise `db_recovery_file_dest_size`, or back up and delete archived logs to free FRA space. The stuck session continues on its own once archiving resumes.

RMAN restore fails and Ansible shows only "censored"

SYMPTOM *ASYNC FAILED* on the destination, followed by *censored: the output has been hidden due to the fact that 'no_log: true' was specified.*

CAUSE Expected for PDB→PDB — the restore runs with `no_log` because its connect string carries the SYS password. The underlying error is in the restore log, not the Ansible output.

FIX Read the newest `restore_<level>_*.log` in `{{ m5_workdir }}/log` on the destination. During testing it showed `ORA-17502: ksfdcre:4 Failed to create file`, on a different datafile each retry; the root cause wasn't confirmed. Check FRA and ASM space and the ASM alert log, then follow the full reset above before retrying.

Complete file index

Every file in the collection, for quick lookup.

Playbook, vars and shared files

PATH	PURPOSE
`playbooks/m5_migration.yml`	The migration playbook — inventory check, then one play per stage, one tag per stage, embedded runbook comments
`playbooks/vars/m5_migration/mig_vars.yml`	Single migration variables file — see Variables file
`playbooks/vars/m5_migration/mig_vault.yml`	ansible-vault encrypted — SYS/SYSTEM passwords
`playbooks/inventory.yml`	Shared PowerODBA inventory — migration hosts in `[mig_source]`, `[mig_destination]`, `[mig:children]` and `[mig:vars]`
`playbooks/ansible.cfg`	Shared PowerODBA settings — unchanged by the migration
`collections/requirements.yml, galaxy.yml`	List `ansible.posix`, needed for the SSH key setup in prechecks
`.gitignore`	Keeps `roles/mig_prechecks/files/*.zip` (DBMIG, patch zips) out of git
`docs/Readme-M5-Migration.md`	Migration guide — prerequisites, quick start, runbook, troubleshooting

roles/mig_prechecks/

PATH	PURPOSE
`README.md, meta/main.yml`	Role summary and Galaxy metadata, in PowerODBA's role format
`defaults/main.yml`	Disk/memory thresholds, standalone bugfix patch IDs (35564253, TDE, standby), min compatible version, TDE marker path, system_usr
`tasks/main.yml`	Sequencing spine — idempotency gate, then 9 includes in order
`tasks/os_checks_source.yml`	See role reference above
`tasks/os_checks_dest.yml`	See role reference above
`tasks/patch_apply_source.yml`	See role reference above
`tasks/patch_apply_dest.yml`	See role reference above
`tasks/ssh_equivalence.yml`	See role reference above

tasks/db_prepare_source.yml	See role reference above
tasks/db_prepare_dest.yml	See role reference above
tasks/deploy_dbmig.yml	See role reference above
tasks/generate_report.yml	See role reference above
templates/dbmig_driver.properties.j2	See role reference above
files/ (zips not tracked)	Manually staged: DBMIG_5.1.zip only — patch 38170982 is checked via opatch, never staged or applied by this collection

roles/mig_rman_L0_L1/

PATH	PURPOSE
README.md, meta/main.yml	Role summary and Galaxy metadata, in PowerODBA's role format
defaults/main.yml	rman_level (set per play) and restore_cmdfile (runtime)
tasks/main.yml	Sequencing spine — timeout resolution, L0 idempotency gate, backup block (source), restore block (destination), done-file write, completion message
tasks/check_lockfile.yml	See role reference above
tasks/run_backup.yml	See role reference above
tasks/verify_backup_log.yml	See role reference above
tasks/get_restore_cmdfile.yml	See role reference above
tasks/sync_backup_pieces.yml	See role reference above
tasks/run_restore.yml	See role reference above
tasks/verify_restore_log.yml	See role reference above

roles/mig_rman_L1F/

PATH	PURPOSE
README.md, meta/main.yml	Role summary and Galaxy metadata, in PowerODBA's role format
defaults/main.yml	rman_level, impdp_run_mode, and runtime-populated expdp_dumpfile / restore_cmdfile
tasks/main.yml	Sequencing spine — 4 stages (backup/restore/import/postchecks) with stage banners, plus the separate tag-gated go-live include

tasks/check_lockfile.yml	See role reference above
tasks/run_backup.yml	See role reference above
tasks/verify_backup_log.yml	See role reference above
tasks/get_restore_cmdfile.yml	See role reference above
tasks/sync_backup_pieces.yml	See role reference above
tasks/capture_expdp_file.yml	See role reference above
tasks/run_restore.yml	See role reference above
tasks/verify_restore_log.yml	See role reference above
tasks/pre_import_checks.yml	See role reference above
tasks/run_impdp.yml	See role reference above
tasks/verify_import.yml	See role reference above
tasks/verify_migration_correctness.yml	See role reference above
tasks/set_tablespaces_readwrite.yml	See role reference above
tasks/restore_source_tablespaces.yml	See role reference above
tasks/recompile_objects.yml	See role reference above
tasks/gather_stats.yml	See role reference above
tasks/validate_migration.yml	See role reference above
tasks/drop_restore_point.yml	See role reference above